

高性能计算机体系结构模型

高性能计算机体系结构课程汇报（最终版）

宋震，苏晓，郭冠男，张泰然，刘旭昭

2025-01-07

目录

1 概述	3
1.1 分类原则	3
2 SIMD 架构结构介绍	4
2.1 SIMD 的概念	4
2.2 向量流水计算机	4
2.2.1 向量化与流水线化的概念	4
2.2.2 向量流水计算机的实现	5
2.3 处理器阵列 SIMD	6
2.3.1 处理器阵列的概念	6
2.3.2 处理器阵列的实现	6
2.4 SIMD 在现代处理器中的应用	6
2.5 GPU	7
2.5.1 GPU 架构更迭	7
2.5.2 Tensor Core	9
2.6 NPU-IANUS	11
2.6.1 统一的内存架构	11
2.6.2 PIM 访问调度 (PAS)	12
2.6.3 高效的多头注意力 (MHA) 调度	12
2.6.4 内存地址映射与行列优化	12
2.6.5 PIM 控制单元 (PCU) 与 PIM 内存控制器 (PIM MC)	13
2.6.6 网络-on-chip (NoC)	13
2.6.7 资源优化与并行性提升	14
2.6.8 任务映射与调度	14
3 MIMD 架构结构介绍	15
3.1 共享存储模型 SMP 与 CC-NUMA	15
3.1.1 SMP (对称多处理器)	15
3.1.2 CC-NUMA (缓存一致非一致存储访问)	16
3.1.3 SMP 与 CC-NUMA 的对比	16

3.2	PVP (并行向量处理器)	16
3.3	MPP (大规模并行处理器)	21
3.4	Cluster (集群)	21
3.5	Compute Express Link (CXL)	22
3.5.1	CXL 提出的背景	22
3.5.2	CXL 的版本与历史沿途	23
3.6	集群虚拟化技术	24
3.6.1	虚拟化对 HPC 的影响	24
3.6.2	常见的虚拟化技术	25
3.7	Cluster 与 MPP 硬件架构比较	26
3.7.1	Cluster 架构硬件架构	26
3.7.2	MPP 架构硬件架构	28
3.7.3	Cluster 与 MPP 对比	29
3.7.4	总结	30
4	总结	30
5	分工情况以及参考文献	31
5.1	小组分工情况	31

1 概述

自弗林分类法 [7] 后, 有关高性能计算机体系结构的讨论层出不穷。在弗林分类法中, 计算机体系结构被分为四个大类: SISD(Single Instruction Stream, Single Data Stream)、SIMD(Single Instruction Stream, Multiple Data Stream)、MISD(Multiple Instruction Stream, Single Data Stream)、MIMD(Multiple Instruction Stream, Multiple Data Stream) 的。其中 SISD, 即单指令流、单数据流计算机, 常用于对计算效率要求不高的低端工业控制芯片, 或仅以一个计算单元的身份作为组成高性能计算机的组成元件, 因而无缘于本文的讨论。

而关于 MISD, 即单指令流多数据流计算机, 目前主要存在两种不同的声音。第一种声音认为, 多指令流单数据流仅在理论上可行而缺乏现实世界中的实例 [7]。另一种声音认为, 诸如脉动阵列 (Systolic Array) [12] 等矩阵运算的专用 AI 加速芯片, 由于其中硬件编码实现了一些固定的指令序列, 而数据从中并行得流入, 因而在某种程度上被认为是 MISD 计算机的实例 [11]。但无论出于那种理由, 我们都不难发现, MISD 难以实现一个易于编程的图灵完备系统, 因此也无法承担起高性能计算机整体架构的重任, 因而同样无缘于文本的讨论。

本文核心关注高性能计算机的整体架构的分类以及各分类间的联系, 具体而言, 本文将依照弗林分类法中的 SIMD 以及 MIMD 两种体系结构并对其展开讨论。为了对比不同体系结构, 我们收集了 40 年以来在高性能计算机领域具有一定代表性的指令集、芯片或超算等共 15 个案例, 试图从特殊到一般地介绍这些不同的高性能计算机体系结构的共性与特性, 进而力求给出我们自己对这些体系结构的一些对比、看法以及历史性的评述。

1.1 分类原则

高性能计算机先驱 Seymour Roger Cray[4] 曾经说: “If you were plowing a field, which would you rather use: two strong oxen or 1024 chickens?” (如果你要耕一块地, 你愿意用 2 头牛还是 1024 只鸡呢?)。Cray 使用这句话反驳那些 MIMD 的拥护者, 企图向他们论证自己的 SIMD 设计能更有效地解决高性能计算中的问题。

随着 MPP (Massively Parallel Processing) 以及节点集群 (Cluster) 在高性能计算中应用的日渐普及, 越来越多的人逐渐认可了 Cray 观点的对立面。尽管如此, SIMD 的高性能设计仍然并非一无是处, 很多在个人计算机以及小型工作站上使用的商用 CPU (Central Processing Unit) 中都采用了的向量指令集作为其指令集的拓展, 其中也体现了一些 SIMD 的设计理念 [8]。至今更多研究者已经不再纠结于 SIMD 与 MIMD 的二择问题, 而侧重关注于如何将 SIMD 与 MIMD 结合在一起以在“后摩尔定律时代”尽可能进一步提升计算机系统的性能 [9]。

因此, 我们在本文中遵循了以下的分类原则, 着重介绍了几类体系结构 (详见图 1)。从指令流的角度我们首先将高性能计算机体系结构划分为 SIMD 和 MIMD 两类, 由于单机可扩展能力的限制, 在 Top500 超级计算机列表中, 目前绝大多数大多数超级计算机从顶层架构的角度来看均属于 MIMD 计算机 [26]。图 2 为 Top500 超级计算机中 MPP 与 Cluster 两类顶层架构的分布情况。图中绿色区域表示采用了 MPP 架构的超级计算机, 蓝色的区域表示采用了 Cluster 架构的超级计算机, 而每部分的面积正比于该计算机的浮点运算性能 (单位 GFLOPS)。图中不难看出, 在 Top500 列表中, 大多数计算机采用了 Cluster 的架构, 但其中性能最高的实例中仍然以 MPP 架构

为主。

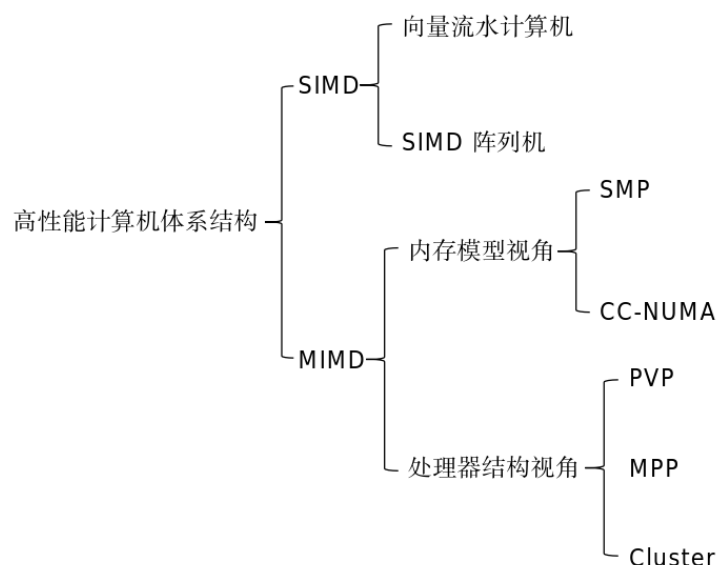


图 1: 高性能计算机体系结构分类思路

2 SIMD 架构结构介绍

2.1 SIMD 的概念

SIMD 是单指令多数据的缩写，它是一种并行计算技术，可以使得处理器在一个时钟周期内通过一条指令同时对多个数据进行操作。与传统的 SISD（单指令单数据）不同，SIMD 允许在同一指令下并行处理多个数据元素，极大地提高了计算效率，特别适用于大量相同运算的任务，如图像处理、科学计算、信号处理等。

与 SISD 相比，SIMD 架构因其高效性、能效和广泛的适用范围而备受青睐。通过并行化计算任务，SIMD 能够显著减少程序的执行时间，这使得它在处理大量数据时比传统的串行计算方式更加高效。此外，SIMD 架构在较低的能耗下能够处理更多的数据，展现出卓越的能效比。这种架构特别适用于多媒体应用、科学计算、机器学习中的矩阵运算等领域，因为它能够同时对多个数据执行相同的操作，从而加速这些领域的计算过程。

2.2 向量流水计算机

2.2.1 向量化与流水线化的概念

向量流水计算机是结合了向量处理和流水线技术的一种优化方案。向量化处理指的是对多个数据进行统一操作，在一条指令下同时处理多个数据元素（如向量加法、乘法等）。流水线技术将复杂的计算过程分成多个阶段，每个阶段可以同时处理不同的数据。在每个时钟周期内，不同的数据会在流水线的不同阶段被处理，最大化资源的使用。在这种架构下，SIMD 计算单元不仅并行处理多个数据元素，还通过流水线方式进行分阶段处理。这种设计可以充分利用硬件并行性，在避免了数据间依赖的前提下，可以大幅提升批量数据的处理速度。

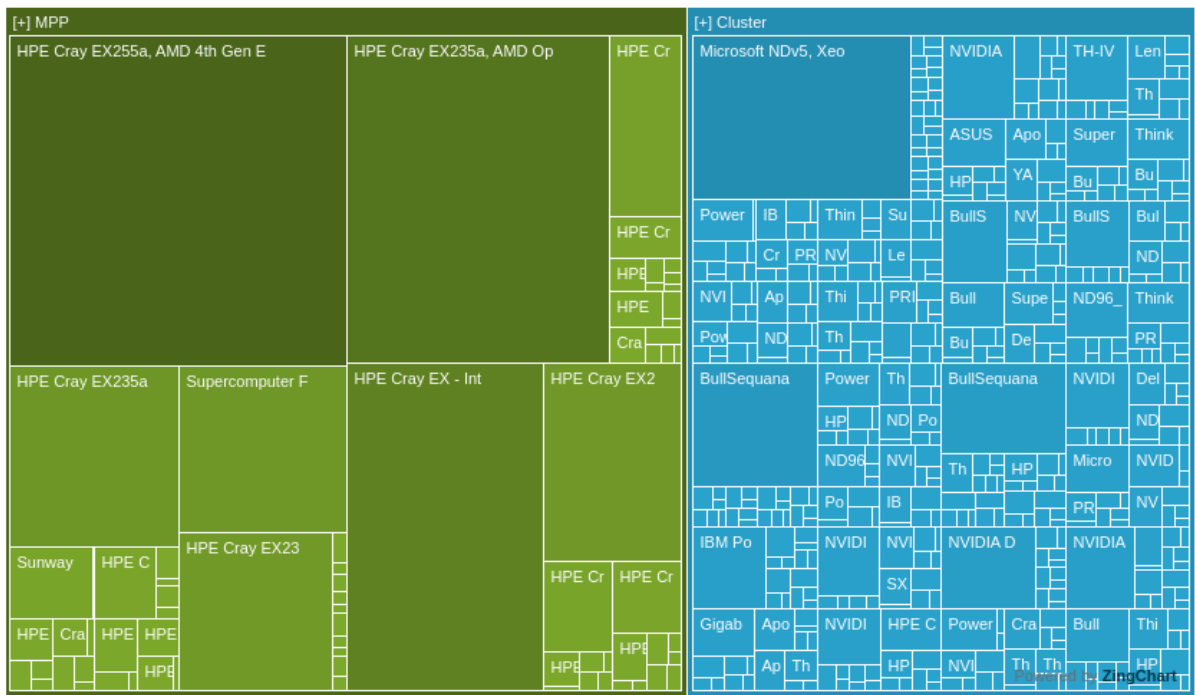


图 2: Top500 超级计算机体系结构占比情况

2.2.2 向量流水计算机的实现

以向量流水线为例，向量流水计算过程将向量的计算过程划分为多个阶段，每个阶段可以并行执行。从编程的角度，向量运算通常可以被理解为三个阶段串行执行的结果。阶段 1：加载数据，即将数据加载到向量寄存器。阶段 2：运算阶段，即执行运算，如加法、乘法等。阶段 3：存储结果，即将结果写回内存或输出寄存器。在流水线中，向量流水机可以按照上述三个阶段划分流水（详见图 3）。当有多组向量数据需要进行处理时，这种流水线技术能够确保处理器在每个时钟周期内都有数据在各级流水线中经过，从而提升计算吞吐率。

具体的流水线计算机可能会采用更多级的流水策略以进一步提高系统的并行度。Shen[15] 给出了设计向量流水线计算机的一些常见思路，其中提到了五级流水、竞争消解以及 tomasulo、记分板等流控制等相关技术。由于处理机在单周期内可以同时并行处理处于不同阶段的多条命令，多个数据在不同阶段同时执行，因而流水向量机的设计实现了事实上的指令级并行，显著提高吞吐量。通过流水线，处理器能够保持高效运作，等待时间和闲置周期减少，数据处理时延降低。

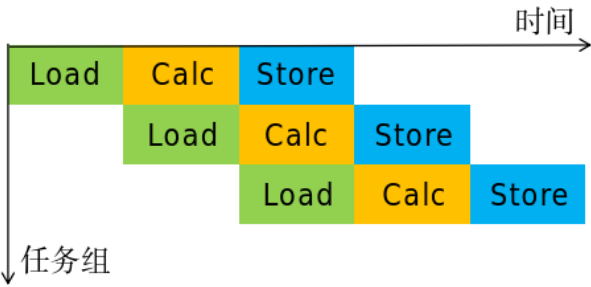


图 3: 流水线加速原理示意图

2.3 处理器阵列 SIMD

2.3.1 处理器阵列的概念

处理器阵列 SIMD 架构是另一种 SIMD 模式下的处理器加速设计 [6]。在处理器阵列 SIMD 架构中，多个处理单元 (PU) 以二维或一维阵列形式排列。每个 PU 都可以执行相同的操作，但每个 PU 在同一时刻处理不同的数据。与向量流水机相比，处理器阵列机中引入了更多的 PU，各个 PU 之间并行工作，从而实现吞吐量的提升。所有参与执行同一任务的 PU 依赖一套统一的指令流的控制进行数据的批量计算。

因此，处理器阵列在执行指令时通常具有以下三个阶段：数据分发、指令同步以及结果合并。在数据分发阶段，输入数据被划分成多个块，并分别送到不同的处理单元。指令同步可以让所有处理单元并行执行相同的指令，但它们作用于不同的数据元素。在各 PU 完成相应命令后，结果合并阶段将各个处理单元的结果合并到一起，从而形成最终的计算结果。

与向量流水机相似，由于多个处理单元可以同时处理大量数据，因此处理器阵列也能够胜任很多需要大规模并行计算的任务。从设计的角度而言，相较于向量流水机基于流水阶段的并行，处理器阵列中被使用的 PU 的数量可以根据具体数据形态的需要进行调整，系统的可拓展型也更为灵活。但多 PU 并行的结构使得负载均衡问题需要额外的考虑，这种额外的考虑会增加软件与硬件的复杂性。如果 PU 调度实现不能充分利用各个 PU 的计算能力，会导致 PU 空闲，从而更容易导致计算资源的浪费。

2.3.2 处理器阵列的实现

本文主要考虑两种处理器阵列的硬件实现模式：借助 FPGA (Field-Programmable Gate Array) 的 PU 实现以及 GPU 风格的 PU 实现。

GPU 的架构本质上是一个处理器阵列，其中有上千个计算单元，每个单元都执行相同的指令，处理不同的数据。这种架构适合用于处理图形渲染和科学计算等应用。GPGPU 中诸如 CUDA 以及 OpenCL 等 SIMT 编程模型为 GPU 上的并行任务执行提供了统一且便利的编程模型。与此同时，基于 warp/wavefront 的线程簇调度也能在一定程度上避免 SM (Streaming Multiprocessor) 闲置导致的资源浪费。

FPGA 是一种可编程的阵列结构，它使用 LUT (Look Up Table) 技术实现了对组合逻辑电路的模拟。由于其内在天然的并行性，FPGA 常被用于实现处理器阵列 SIMD (Single Instruction Multiple Threads) 中的 PU 阵列。[14] 中列举了一些 FPGA 在 AI 并行加速训练中的应用实例。

Nurvitadhi[14] 认为，在很多并行计算领域中，FPGA 有着并不逊色于 GPU 的性能，但在工业实践中却更少被人提及。本文认为，GPU 的广泛普及得益于其在软件层面上的统一编程模型，这种软件层面的统一为 GPU 的商业化推广提供了便利。

2.4 SIMD 在现代处理器中的应用

Intel AVX (Advanced Vector Extensions) 指令集 [1] 是 Intel 在其处理器中引入的一系列 SIMD 指令集的扩展，旨在提高处理器在处理图形、音频、视频、科学计算和金融分析等需要大量浮点运

算的应用时的性能。AVX 指令集首次出现在 Intel 的 Sandy Bridge 微架构处理器中，并于 2011 年随第二代 Core 处理器系列一同发布。

与 Intel AVX 拓展指令集类似，ARMv6 架构中引入了名为 ARM Neon 的向量指令集拓展 [2] 以实现并行运算功能的加速。相较于 AVX 中提供的 256 位的向量寄存器，ARM Neon 提供了 128 位长度的向量寄存器。由于 ARM 指令集常用于低功耗移动设备以及嵌入式设备中，因此在环境的制约下一般无法兼顾并行需求可能带来的功耗提升。与适用于桌面端以及工作站的 x86_64 架构相比，ARM 架构有着更加明显的低功耗需求，因此高性能并不是其首要考虑。

相较于 SISD 的处理器结构，SIMD 技术通过在单一指令下并行处理多个数据元素，提高了运算效率。向量流水计算机以及处理器阵列 SIMD 是两种不同的借助 SIMD 提高计算吞吐率的思路。向量流水计算机结合了向量化处理和流水线技术，能够在多个阶段并行处理不同数据，借助阶段间的并行提高了吞吐量和效率。而处理器阵列 SIMD 通过将多个 PU 排列成阵列，借助 PU 间的并行提升 SIMD 任务的执行效率。我们可以看到两种设计思想在当今的商用 CPU 中都有着广泛体现。

2.5 GPU

2.5.1 GPU 架构更迭

Tesla 架构 (2008 年)

- 架构特点：是英伟达第一个实现统一着色器模型的微架构，支持 C 语言编程，引入了单指令多线程 (SIMT) 执行模型，大大提高了 GPU 的并行处理能力。
- 代表产品：经典型号是 G80。
- 主要改进：相较于之前的架构，统一着色器模型使得 GPU 能够更加灵活地分配资源，根据不同的图形渲染任务需求，动态地将处理单元分配给顶点、像素、几何等不同类型的着色器，从而提高了整体的渲染效率。SIMT 执行模型则允许在同一指令下并行处理多个数据元素，进一步挖掘了 GPU 的并行计算潜力，为后续的通用计算和深度学习应用奠定了基础。

Fermi 架构 (2010 年)

- 架构特点：首款支持与共享存储结合纯 cache 层次的 GPU 架构，也是支持 ECC 的 GPU 架构。
- 代表产品：如 GF100 等。
- 主要改进：引入了共享存储和 cache 层次结构，使得 GPU 在处理复杂的图形和计算任务时，能够更高效地管理数据，减少数据访问延迟，提高数据复用率，从而提升了性能。支持 ECC (Error Correcting Code) 能够检测和纠正内存中的数据错误，提高了系统的可靠性和稳定性，这对于一些对数据准确性要求较高的科学计算和专业图形应用非常重要。

Kepler 架构 (2012 年)

- 架构特点：相较于 Fermi 更快，效率更高，性能更好，是第一款支持 CUDA 的 GPU 架构。
- 代表产品：基于 GK104 核心的 GTX680 等。

- 主要改进：采用了全新的 SMX 单元，相比之前的 SM 单元有较大改动，内部的 CUDA Core 数量大幅增加，同时核心频率也显著提高，使得 GPU 的计算能力得到了大幅提升。引入了 GPU Boost 技术，这是一种动态超频技术，能够根据 GPU 的负载情况自动调整核心频率，在不超过功耗限制的前提下，尽可能地提高性能，进一步提升了 GPU 的能效比。此外，在制造工艺上采用了 28 纳米工艺，使得晶体管数量和核心面积都有所优化，降低了功耗和成本。

Maxwell 架构 (2014 年)

- 架构特点：引入了立体像素全局光照 (VXGI) 技术。
- 代表产品：如 GTX980 等。
- 主要改进：VXGI 技术的引入首次让游戏 GPU 能够提供实时的动态全局光照效果，大大提升了游戏画面的真实感和视觉效果。在架构设计上，对 CUDA Core、纹理单元、缓存等进行了优化，提高了每瓦性能，使得 GPU 在相同功耗下能够提供更强大的性能，或者在相同性能下功耗更低。此外，还对多 GPU 并行技术进行了改进，提升了多卡协同工作的效率。

Pascal 架构 (2016 年)

- 架构特点：将处理器和数据集成在同一个程序包内，以实现更高的计算效率。
- 代表产品：1080 系列、1060 系列等。
- 主要改进：采用了更先进的 16 纳米 FinFET 制造工艺，提高了晶体管的性能和能效。在架构上，通过将处理器和数据集成在一起，减少了数据传输延迟，提高了数据访问速度，从而提升了计算效率。此外，还引入了 NVLink 技术，大大提高了多 GPU 之间的通信带宽和效率，使得多 GPU 系统在处理大规模数据和复杂计算任务时能够更加高效地协同工作。

Volta 架构 (2017 年)

- 架构特点：配备 640 个 Tensor Core，每秒可提供超过 100 兆次浮点运算 (TFlops) 的深度学习效能。
- 代表产品：如 V100 等。
- 主要改进：专注于深度学习和人工智能应用，引入的 Tensor Core 专门用于加速深度学习中的张量计算，如矩阵乘法和卷积运算，相比传统的 CUDA Core，在处理深度学习任务时具有更高的效率和性能，使得 GPU 在人工智能领域的应用得到了极大的拓展，为深度学习模型的训练和推理提供了强大的计算支持。

Turing 架构 (2018 年)

- 架构特点：配备了名为 RT Core 的专用光线追踪处理器。
- 代表产品：如 RTX2080 等。
- 主要改进：实时光线追踪技术是 Turing 架构的一大亮点，RT Core 能够以高达每秒 10Gi-gaRays 的速度对光线和声音在 3D 环境中的传播进行加速计算，从而实现了在游戏、动画等领域中实时模拟光线的传播、反射、折射等物理现象，为图形渲染带来了更逼真的光影效果。同时，还引入了深度学习超采样 (DLSS) 技术，通过人工智能算法提高图像的渲染质量，在不降低图像质量的前提下，提高渲染速度，降低了对硬件性能的要求。

Ampere 架构 (2020 年)

- 架构特点：在计算能力、能效和深度学习性能方面都有重大提升，引入了第三代 Tensor Core。
- 代表产品：如 A100 等。
- 主要改进：采用了多个流多处理器 (SM) 和更大的总线宽度，提供了更多的 CUDA Core 和更高的频率，进一步提高了计算能力。第三代 Tensor Core 具备更强大的深度学习计算性能，支持更多的数据类型和更复杂的计算操作，能够更好地满足大规模深度学习模型的训练和推理需求。此外，该架构的 GPU 还具有更高的内存容量和带宽，适用于大规模的数据处理和机器学习任务。

Hopper 架构 (2022 年)

- 架构特点：以美国计算机领域的先驱科学家 Grace Hopper 的名字命名，拥有强大的 Tensor Core，适用于大规模 AI 模型训练和推理。
- 代表产品：如 H100 等。
- 主要改进：对 Tensor Core 进行了进一步优化，提高了其在深度学习任务中的性能和效率。采用了更先进的制造工艺和架构设计，提升了 GPU 的整体性能和能效比。此外，还引入了一些新的技术和功能，如异步数据传输、多实例 GPU 等，进一步提高了 GPU 在不同应用场景下的灵活性和适应性。

Blackwell 架构 (2024 年)

- 架构特点：将两个芯片结合在一起，采用统一内存架构 + 双芯配置，集成 2080 亿颗晶体管，采用定制台积电 4NP 工艺。
- 代表产品：如 B200、GB200 等。
- 主要改进：相比上一代 Hopper 架构，Blackwell 集成了两个 die，面积变大，晶体管数量大幅增加，从而提供了更强大的计算能力。拥有第二代 Transformer 引擎，将新的微张量缩放支持和先进的动态范围管理算法与 TensorRT-LLM 和 NeMo Megatron 框架结合，使 Blackwell 具备在 FP4 精度的 AI 推理能力，可支持 2 倍的计算和模型规模，能在将性能和效率翻倍的同时保持混合专家模型的高精度。配备第五代 NVLink，为每个 GPU 提供 1.8TB/s 双向带宽，支持多达 576 个 GPU 间的无缝高速通信，适用于复杂大语言模型。还包括 RAS 引擎，可利用基于 AI 的预防性维护来进行诊断和预测可靠性问题，最大限度延长系统的正常运行时间，提高大规模 AI 部署的弹性，并降低运营成本。此外，具有先进的机密计算功能可保护 AI 模型和客户数据，以及解压缩引擎，支持最新格式，加速数据库查询，以提供数据分析和数据科学的最高性能。

2.5.2 Tensor Core

Tensor Core (张量核心) 是现代 GPU 中用于加速深度学习、人工智能 (AI) 和高性能计算 (HPC) 任务的重要硬件单元。Tensor Core 技术是专门为矩阵运算设计的，它通过并行计算和优化运算流程，极大地提升了机器学习、深度学习等计算密集型任务的性能。以下是现有 GPU 中的 Tensor Core 技术及其应用的重点介绍。

Tensor Core 的工作原理 Tensor Core 的设计专注于高效的矩阵乘法，特别是在深度学习中的应用。深度学习训练中最常见的操作是矩阵乘法（例如在前向传播和反向传播过程中）。Tensor Core 通过优化这些操作，显著提升了计算效率。它可以在一个时钟周期内并行执行多个浮点操作，从而大幅度提高了计算吞吐量。

- 矩阵乘法加速：Tensor Core 通常执行的是矩阵乘法（GEMM），尤其适用于深度学习中的卷积神经网络（CNN）等模型。通过使用混合精度计算，Tensor Core 可以在每个时钟周期内执行更多的乘法操作。

支持不同精度 Tensor Core 能够处理不同的数据精度，包括：

- FP16：浮点 16 位，适用于训练加速。
- FP32：浮点 32 位，适用于更高精度的计算。
- TF32：Tensor Float 32，一种 NVIDIA 定义的新的浮点精度，提供了 FP32 和 FP16 之间的性能和精度平衡。
- Bfloat16：一种 16 位浮点精度，广泛用于机器学习训练，特别是在 Google 的 TPU 中。

高度并行化计算 并行性：Tensor Core 设计目标之一就是极大地提高计算密度和并行性。在一个时钟周期内，Tensor Core 能够执行多个浮点或整数运算操作（如矩阵乘法、加法等）。这使得它能够在处理大规模矩阵运算时显著提高吞吐量。矩阵运算加速：Tensor Core 专门为加速矩阵乘法（GEMM）和卷积运算等深度学习中常见的计算任务设计。由于这些运算通常需要处理大量的并行计算，Tensor Core 通过并行化处理加速了深度学习模型的训练和推理过程。

多级并行性 多级流水线：Tensor Core 的内部设计支持多级流水线，每个 Tensor Core 能够同时进行多个不同类型的计算任务。这种多级流水线设计使得 Tensor Core 能够高效地利用 GPU 的计算资源。数据流控制：Tensor Core 在数据流的控制方面也进行了优化，能够更高效地调度数据，确保数据在执行过程中不会成为瓶颈。

低延迟与实时推理 低延迟加速推理：Tensor Core 不仅适用于大规模训练任务，也非常适合低延迟的推理任务。深度学习推理要求快速响应时间，而 Tensor Core 能够通过快速处理矩阵运算，显著降低推理延迟，提供实时的 AI 推理能力。AI 推理加速：Tensor Core 在处理大规模推理任务时，特别是在推理模型部署（如智能设备、云推理）中，提供了高效的加速支持。

支持多个深度学习框架 框架兼容性：Tensor Core 能够与主流的深度学习框架（如 TensorFlow、PyTorch、Caffe 等）紧密集成，并通过优化库（如 CUDA、cuDNN）来提升性能。这使得开发者可以无缝利用 Tensor Core 进行深度学习任务的加速。自动优化：Tensor Core 通过 NVIDIA 的深度学习加速库自动进行优化。开发者可以直接使用这些库，无需手动调整低级硬件细节，Tensor Core 会自动识别并优化计算任务。

适应不同硬件平台 多 GPU 支持：Tensor Core 不仅在单个 GPU 上有效，也支持多 GPU 环境下的分布式训练和推理。在多 GPU 系统中，Tensor Core 能够通过 NVIDIA NVLink 等高速互联技术协同工作，进一步提升计算能力。跨架构支持：Tensor Core 能够适应不同 NVIDIA GPU 架构（如 Volta、Turing、Ampere、Hopper 等），并根据硬件的不同特性进行优化。因此，Tensor Core 在不同型号的 GPU 中都能够提供高效的 AI 加速支持。

专为 AI 和深度学习优化 卷积操作优化：深度学习中，卷积操作是非常常见的计算任务。Tensor Core 通过专门优化卷积运算，使得其在卷积神经网络（CNN）和其他深度学习任务中的表现尤为

突出。加速神经网络训练和推理：Tensor Core 通过加速大规模矩阵乘法操作，显著提升了深度学习训练和推理过程中的性能。例如，Tensor Core 可以大幅减少训练神经网络所需的时间，使得 AI 开发者能够更快地进行模型优化和迭代。

高吞吐量与高效能 计算吞吐量：Tensor Core 通过优化硬件设计来大幅度提高矩阵运算的吞吐量。在 NVIDIA 的 Volta、Turing、Ampere 等架构中，Tensor Core 能够同时执行多个乘法加法操作（MAC，Multiply-Accumulate），提供非常高的计算吞吐量。例如，Volta 架构中的每个 Tensor Core 能够在 1 个时钟周期内执行 32 个 FP16 的乘法加法操作。性能提升：在深度学习任务中，Tensor Core 通过加速常见的计算任务（如矩阵乘法、卷积）来显著提升训练和推理的速度。例如，在 NVIDIA 的 A100 GPU 中，Tensor Core 可以提供每秒数十万亿次（TFLOPS）的计算能力。

高效的内存访问 内存访问优化：Tensor Core 在设计时也考虑到了内存访问的效率。在 GPU 中，内存带宽通常是计算性能的瓶颈之一。Tensor Core 通过高效的内存访问模式（例如在共享内存中的数据复用）来优化内存带宽使用，从而提高整体的计算性能。内存带宽优化：通过减少内存访问次数和增强数据局部性，Tensor Core 能够加速计算过程，特别是在需要处理大规模数据时。

支持多个深度学习框架 框架兼容性：Tensor Core 能够与主流的深度学习框架（如 TensorFlow、PyTorch、Caffe 等）紧密集成，并通过优化库（如 CUDA、cuDNN）来提升性能。这使得开发者可以无缝利用 Tensor Core 进行深度学习任务的加速。自动优化：Tensor Core 通过 NVIDIA 的深度学习加速库自动进行优化。开发者可以直接使用这些库，无需手动调整低级硬件细节，Tensor Core 会自动识别并优化计算任务。

广泛的 AI 应用支持 Tensor Core 被广泛应用于以下领域：自然语言处理（NLP）：Tensor Core 通过加速矩阵运算，大幅提高了 NLP 模型（如 BERT、GPT 系列）的训练和推理速度。计算机视觉：在图像分类、目标检测和生成模型等计算机视觉任务中，Tensor Core 显著提高了卷积神经网络（CNN）的效率。自动驾驶：Tensor Core 支持自动驾驶系统中的深度学习任务，如图像识别、对象检测等，通过加速推理和训练过程，帮助提高自动驾驶系统的响应速度和处理能力。推荐系统：Tensor Core 在大规模推荐系统的训练中起到了关键作用，帮助加速复杂模型的训练，提升个性化推荐的实时性和精度。

总结 Tensor Core 是现代 GPU 中的一个关键技术，其特点不仅仅体现在支持不同的精度计算上，还包括高度的并行计算能力、对深度学习的优化、混合精度支持、高吞吐量、内存带宽优化、低延迟推理、支持多 GPU 和深度学习框架等特性。这些特点使得 Tensor Core 成为深度学习训练和推理中的核心加速单元，极大地提高了 AI 应用的效率和速度。

2.6 NPU-IANUS

IANUS 是一个为处理大规模神经网络，特别是大语言模型（LLMs）优化的统一计算架构，它结合了神经网络处理单元（NPU）和存算一体系统（PIM），旨在通过高效的内存管理、计算资源调度和任务映射技术，实现更高的计算吞吐量和更低的延迟。

2.6.1 统一的内存架构

IANUS 采用统一的内存设计，使 NPU 和 PIM 共享同一物理内存，而不是分开使用不同的内存。这种设计不仅减少了数据复制的需求，还减少了内存访问瓶颈，提高了内存带宽利用率。通过

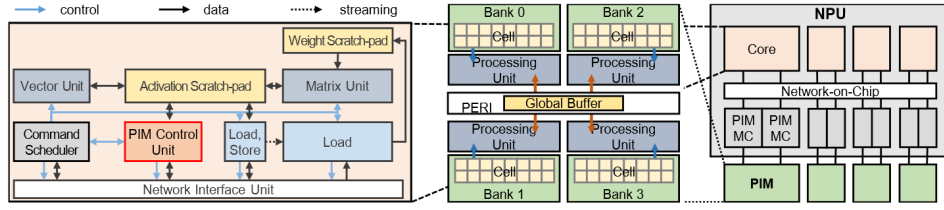


图 4: NPU-IANUS 架构

统一内存架构，PIM 和 NPU 可以在同一内存空间内高效地执行任务，PIM 在执行复杂计算时，能够直接操作内存中的数据，从而减少数据的移动。

2.6.2 PIM 访问调度 (PAS)

PIM 访问调度 (PAS) 是 IANUS 架构中的关键技术，它负责调度 NPU 和 PIM 之间的内存访问任务。PAS 不仅要调度常规的内存命令，还要考虑如何高效地将计算任务映射到 PIM 和 NPU 之间，解决两个计算单元之间的资源竞争问题。

PAS 通过自适应映射算法决定 FC（全连接层）操作的最佳执行单元。该算法基于输入数据的规模、计算单元的执行时间、内存带宽等因素进行决策。

FC 操作的映射：当输入数据较小时，PIM 被优先选用，因为它能够减少内存瓶颈；而对于较大的数据，矩阵单元（MU）则更合适，因为它能够提供更高的计算吞吐量。

2.6.3 高效的多头注意力 (MHA) 调度

在大语言模型中，多头注意力是计算的核心，IANUS 通过两种阶段的调度来优化多头注意力的执行。

总结阶段 (Summarization Stage)：在此阶段，Q、K、V 矩阵通常是进行矩阵乘法的关键操作。IANUS 利用内跨注意力头并行性和注意力头间流水线技术来加速计算。Q、K、V 矩阵的计算由矩阵单元执行，而 DMA 用于高效地从内存加载权重矩阵，减少数据传输时间。

生成阶段 (Generation Stage)：避免了 PIM 和 NPU 的资源冲突，同时确保计算并行性最大化。

2.6.4 内存地址映射与行列优化

IANUS 通过高效的内存地址映射，最大化 PIM 计算的吞吐量。内存地址映射采用了 Row-Channel-Bank-Column 的顺序，确保数据的存储和计算具有最优的局部性和并行性。

- 行地址：保证数据在同一行内进行计算，从而减少行冲突。
- 列地址：通过列地址的映射，确保每一行的所有元素由同一处理单元进行处理。

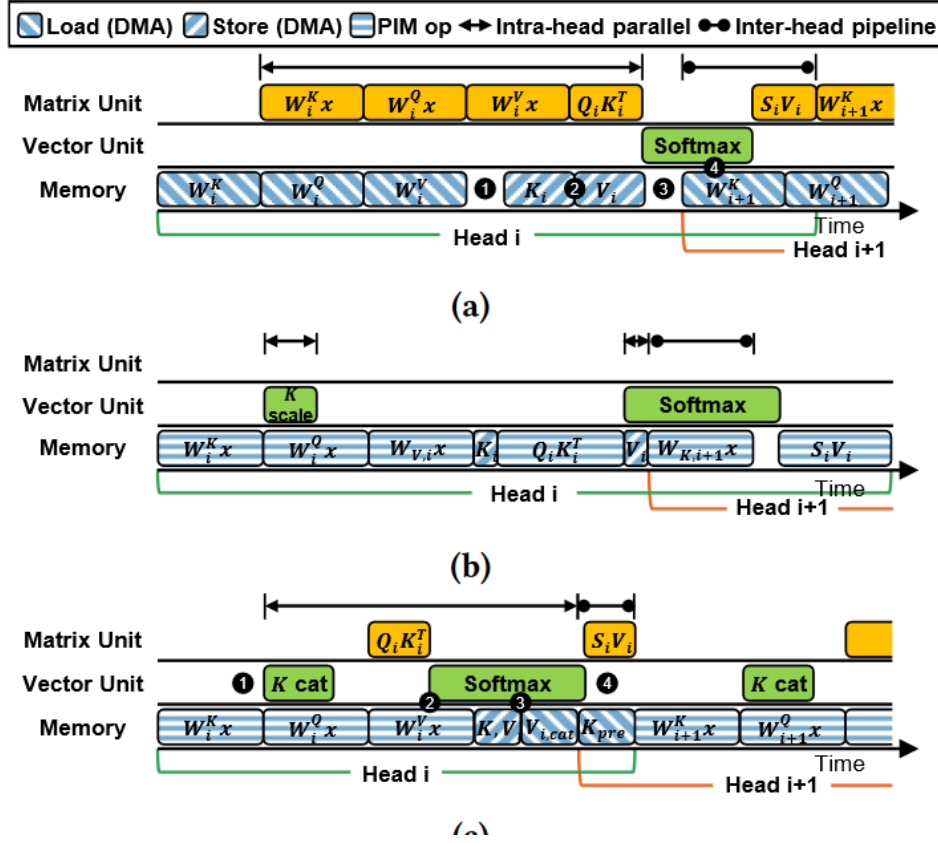


图 5: PIM 访问调度 (PAS)

- Channel 和 bank: 将频道和银行分配给不同的计算单元, 充分利用内存的并行性进行计算, 避免了资源冲突。

2.6.5 PIM 控制单元 (PCU) 与 PIM 内存控制器 (PIM MC)

IANUS 采用了 PIM 控制单元 (PCU) 和 PIM 内存控制器 (PIM MC) 来调度和执行 PIM 命令。PCU 通过将宏 PIM 命令分解为多个微 PIM 命令, 确保 PIM 操作的高效执行。微 PIM 命令通过网络传输到 PIM MC 进行处理。

PIM MC 负责追踪内存状态, 确保各个 PIM 操作和常规内存操作按时序进行调度, 不发生冲突。

2.6.6 网络-on-chip (NoC)

网络-on-chip (NoC) 为 IANUS 架构提供了高带宽的通信机制, 支持 NPU 核心与 PIM 内存控制器之间的全互联通信。这保证了所有核心都可以高效访问任意内存通道, 特别是在 PIM 作为主内存时。

NoC 不仅传输常规内存访问数据, 还传输 PIM 命令, 从 PCU 到 PIM MC 的命令能够进行广播, 从而降低 NoC 带宽的压力, 同时加速 PIM 操作的并行执行。

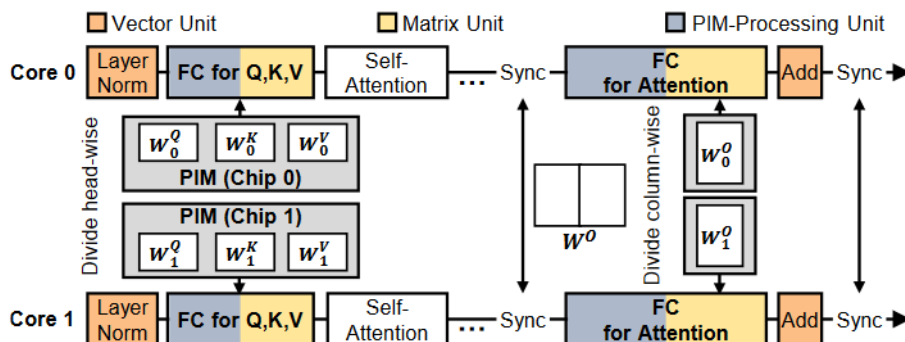


图 6: 多头注意力 (MHA) 调度

2.6.7 资源优化与并行性提升

ANUS 通过精细的资源调度策略，避免了 NPU 和 PIM 之间的资源冲突，并利用 PIM 和 NPU 各自的优势，充分挖掘并行性。

- 内跨注意力头并行性：通过并行计算多个注意力头，提高计算效率。
- 注意力头间流水线：通过流水线技术，将不同阶段的计算任务分配到不同的处理单元，提高执行效率。
- 矢量单元 (VU)：对于一些操作（如残差加法和激活函数），IAUS 通过在矢量单元中执行来减小数据传输的延迟。

2.6.8 任务映射与调度

在执行多头注意力等复杂操作时，IANUS 优化了任务映射与调度，使得计算任务能够根据其需求被动态地映射到 NPU 或 PIM。

通过合理的任务调度，IANUS 能够根据计算需求和内存需求选择最合适的执行单元，避免了计算瓶颈和内存瓶颈。

Intel AMX Architecture

AMX (Advanced Matrix Extensions) 是 Intel 为了加速矩阵运算而引入的一种新的硬件扩展，专门针对深度学习和其他高性能计算任务进行了优化。AMX 架构首次出现在 Intel 的 Sapphire Rapids (第四代 Xeon Scalable) 处理器中。

- 矩阵乘法加速：AMX 的核心目标是通过硬件加速矩阵乘法，尤其是深度学习中的大规模矩阵乘法操作。AMX 通过增加矩阵处理单元，可以比传统的 SIMD 指令集更高效地执行这些任务。
- 硬件支持：AMX 提供了一组新的指令，这些指令直接在硬件中实现矩阵乘法和累加。通过利用 AMX 的硬件加速，计算速度显著提高，尤其在处理大量并行计算时。
- Tile-based Execution：AMX 采用了基于“Tile”的执行模式。每个 Tile 可以看作是一个矩阵块，这使得矩阵运算能够被分割成更小的部分来并行处理，从而提高了性能。
- 高效利用内存带宽：AMX 能够通过局部存储优化内存访问模式，减少内存瓶颈，提高数据流的效率。

Instruction	Description
LD TILECFG	Load Tile Configuration
ST TILECFG	Store Tile Configuration
TDPBF16PS	Dot Product of BF16 Tiles Accumulated into Packed Single Precision Tile
TDPBSSD/TDPBSUD/TDPBUSD/TDPBUUD	Dot Product of Signed/Unsigned Bytes with Dword Accumulation
TILELOADD/TILELOADDT1	Load Tile
TILERELASE	Release Tile
TILESTORED	Store Tile
TILEZERO	Zero Tile

表 1: AMX 指令集

Intel TMUL Architecture

TMUL (Tensor Multiply) 架构是 Intel 为加速 Tensor 计算（尤其是在 AI 和机器学习应用中）的另一种硬件扩展。它主要围绕矩阵和张量运算进行优化，在深度学习等应用中具有重要意义。

Tensor Multiply 指令集：TMUL 架构为 Tensor 计算（如卷积操作）提供了专门的指令集，通过硬件加速深度学习推理和训练中的矩阵和张量运算。

矩阵乘法优化：TMUL 类似于 AMX，也专注于矩阵运算的优化。它优化了数据流、内存访问和并行计算，以加速大规模的矩阵运算和张量计算，尤其适合深度神经网络 (DNN) 计算中的矩阵乘法。

高并行性和吞吐量：TMUL 能够在多个核心和线程中高效并行处理大规模的矩阵运算，提升了吞吐量并降低了延迟。

与 AI 加速兼容：TMUL 架构与 AI 推理和训练工作负载高度兼容，尤其是在利用深度学习框架（如 TensorFlow 或 PyTorch）时，能够显著提高性能。

3 MIMD 架构结构介绍

3.1 共享存储模型 SMP 与 CC-NUMA

在高性能计算 (HPC) 中，SMP（对称多处理）和 CC-NUMA（缓存一致非一致存储访问）是两种常见的内存模型。它们针对多核和多处理器的计算资源进行了优化，以应对不同的计算需求。本文中总结了两种架构的详细介绍及其各自的优缺点。

3.1.1 SMP（对称多处理器）

SMP (Symmetric Multi-Processing) 是一种多处理器体系结构，其中所有处理器共享同一块物理内存。在 SMP 系统中，每个处理器在硬件上是平等的，没有主从之分，所有的 CPU 都可以访问系统中的所有内存资源。这种设计允许系统在多个处理器之间均匀分配任务，从而提高计算效率。

SMP 体系结构是一种对称的共享内存结构。对称性意味着所有处理器平等，能够平等地访问共享内存。共享内存是 SMP 体系结构的另一个关键特征，所有处理器共享同一块物理内存，内存一致性通常由硬件和软件系统共同保证。这种共享内存的特性使得进程间通信非常高效，尤其是多线程程序之间的数据交换。这使得 SMP 有利于完成需要处理器间进行频繁通信的并行任务。

由于 SMP 中各处理机在访问存储器的能力上机会均等，开发者可以利用共享内存轻松实现线程间通信与同步，编程模型比非对称的分布式内存下的编程模型简单。但也正由于各个处理器总是对所有内存地址都具有完全共享的访问能力，随着核心数目的增加，内存带宽会成为系统的主要瓶颈，这使得 SMP 主要适用于核心数较少的系统（几十个核心以内），例如中小型服务器和 workstation。时常被用于多线程计算、图像处理等任务。

3.1.2 CC-NUMA（缓存一致非一致存储访问）

CC-NUMA（Cache-Coherent Non-Uniform Memory Access）是一种多处理器体系结构，其中每个处理器有自己的本地内存，且通过高速网络访问其他处理器的内存。CC-NUMA 架构的关键在于不同处理器访问内存的速度不同，访问本地内存较快，而访问远程内存则较慢。从设计哲学的角度讲，CC-NUMA 考虑到了程序与数据的局部性原理。相比于 SMP，CC-NUMA 架构中的每个处理器访问其自带的本地内存的速度非常快，但访问其他处理器相关的远程内存则有较高的延迟。不同处理器间的缓存一致性通过缓存一致性协议（如 MESI 协议）保证。

CC-NUMA 架构强调局部性，使用该架构的高性能计算机进行程序设计时，应当尽量避免频繁访问远程内存，以减访问远程存储器所带来的延迟。相比于 SMP，CC-NUMA 中的内存由于分别被不同处理器独占，随着处理器数量的增多更不容易成为整个系统的瓶颈，因此 CC-NUMA 架构相较于 SMP 而言具有更好的可拓展性。但这种非对称的设计也使得 CC-NUMA 需要采用比 SMP 更加复杂的编程模型，程序员需要额外注意内存访问局部性以及节点间的通信延迟对程序效率的影响。

CC-NUMA 架构因其优化的内存访问速度和扩展性，在需要处理大规模数据和复杂计算的场景中得到了广泛应用。这种架构常被用于大规模科学计算，例如气候模拟、粒子物理和基因组学等领域，这些领域需要对庞大的数据集进行高速处理和分析。此外，CC-NUMA 也适用于大数据分析 & 数据库应用，它能够有效地支持大规模数据的存储和处理，满足数据中心对高吞吐量和低延迟的需求。

3.1.3 SMP 与 CC-NUMA 的对比

表 2 从内存架构、内存一致性、可拓展性、访存延迟以及应用场景五个角度对比了 SMP 与 CC-NUMA 两种不同内存模型的异同。

图 7 中给出了 SMP 以及 CC-NUMA 的典型结构对比。

3.2 PVP（并行向量处理器）

并行向量处理器的概念

特性	SMP	CC-NUMA
内存架构	所有处理器共享同一物理内存	每个处理器拥有自己的本地内存，通过网络访问其他处理器内存
内存一致性	引入高速缓存时需要采用缓存一致性协议	一般使用诸如 MESI 的缓存一致性协议，需要考虑内存一致性
可拓展性	拓展能力有限，处理器增多时内存带宽容易成为瓶颈	高拓展性，适合大规模计算
访存延迟	处理器间共享内存，访存通信延迟均衡	访问本地内存延迟低，访问远程内存延迟高
应用场景	中小型以及图形工作站	大规模科学计算以及数据分析

表 2: SMP 与 CC-NUMA 的对比

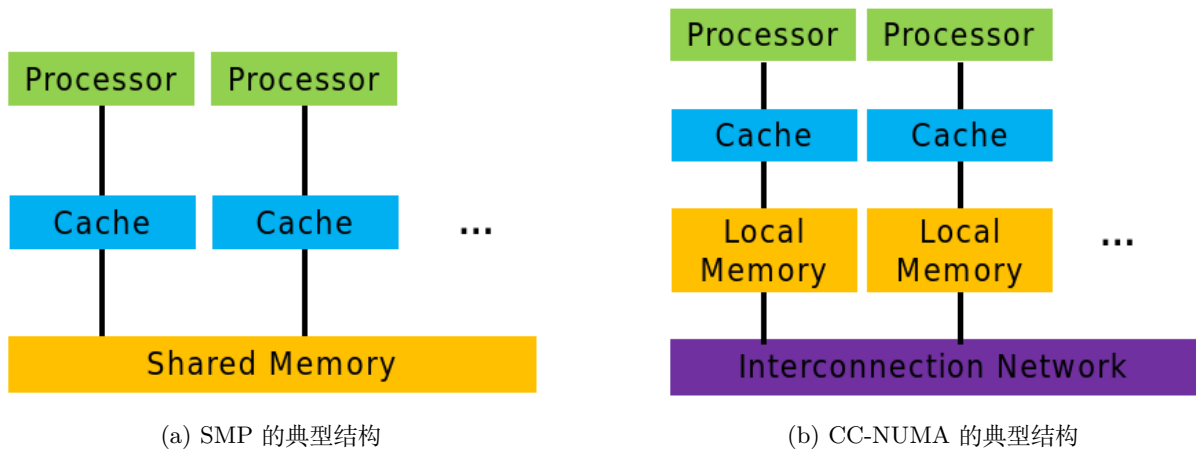


图 7: SMP 与 CC-NUMA 的典型结构对比

并行向量处理机（PVP）由少量的高性能专门设计定制的向量处理器 VP（Vector Processor）组成，每个至少具有 1GFLOPS 的处理能力。如图 8 所示，系统中使用了专门设计的高带宽的交叉开关网络向 VP 连向共享存储模块。并行向量处理机通过向量处理和多个向量处理器并行处理两条途径来提高处理能力。Cray X-MP、Cray T-90、NEC SX-4 和我国的银河 1 号都属于 PVP。

PVP 通常使用定制的高带宽网络将向量处理器连向共享存储器模块。存储器可以以很高的速度向处理器提供数据。例如，在 CrayT-90 中，共享存储器能以 14GB/s 的速率将数据提供给一个处理器。这种机器通常不使用高速缓存，而是使用大量的向量寄存器和指令缓存。

CRAY X-MP

Cray X-MP 是 Cray Research 公司设计、制造和销售的超级计算机，如图 9 所示。它于 1982 年发布，是 1975 年 Cray-1 的“升级版”，并在 1983 年至 1985 年间以四处理器系统性能 800 MFLOPS 成为世界上速度最快的计算机。Cray X-MP 相对于 Cray-1 的主要改进是它是一款共享内存并行向量处理器，这是 Cray Research 推出的第一台此类计算机。它在主机中最多可容纳 4 个 CPU，主机的外观与 Cray-1 几乎完全相同。X-MP 采用新的多处理器架构，在一个机柜中容纳四个处理器。这为并行执行打开了大门，允许模型被划分为许多异步可执行的部分。这种使用模式称为多任务处理，是现代并行计算的先驱。

Cray X-MP CPU 均采用 16 门阵列集成电路。这些电路比 CRAY-1 中使用的电路更快、更密

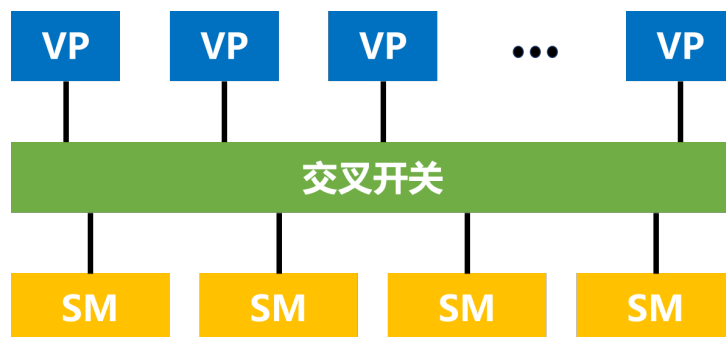


图 8: 并行向量处理器 (PVP) 的典型结构

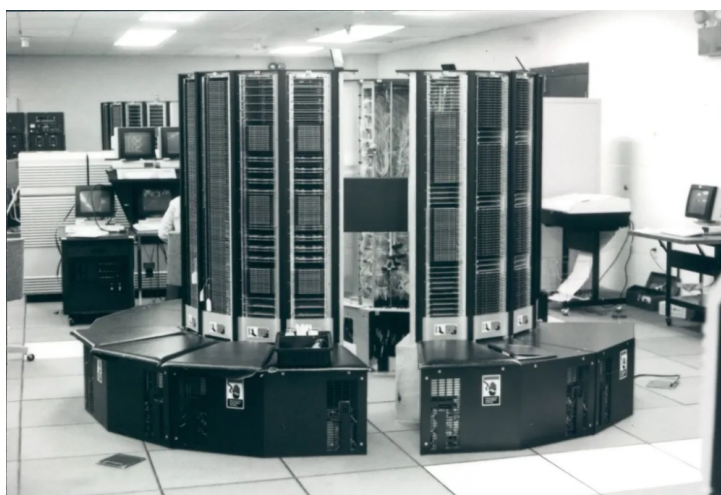


图 9: Cray X-MP

集，时钟周期时间为 9.5 ns，存储体周期时间为 38 ns。外，Cray X-MP 还采用了经过验证的冷却和封装技术，以确保系统的高可靠性。Cray X-MP 每个处理器具有四个并行内存访问端口，结合改进的时钟周期，使其总可用内存带宽超过 CRAY-1 的八倍。Cray X-MP 的高性能在标量模式和向量模式下都得到了体现。标量性能通过更快的时钟、更短的内存访问时间以及更大的指令缓冲区得到提升；而向量性能则通过更快的时钟、并行内存端口以及硬件自动“链接”功能的结合得到了改进。Cray 核心结构如图 10 所示。

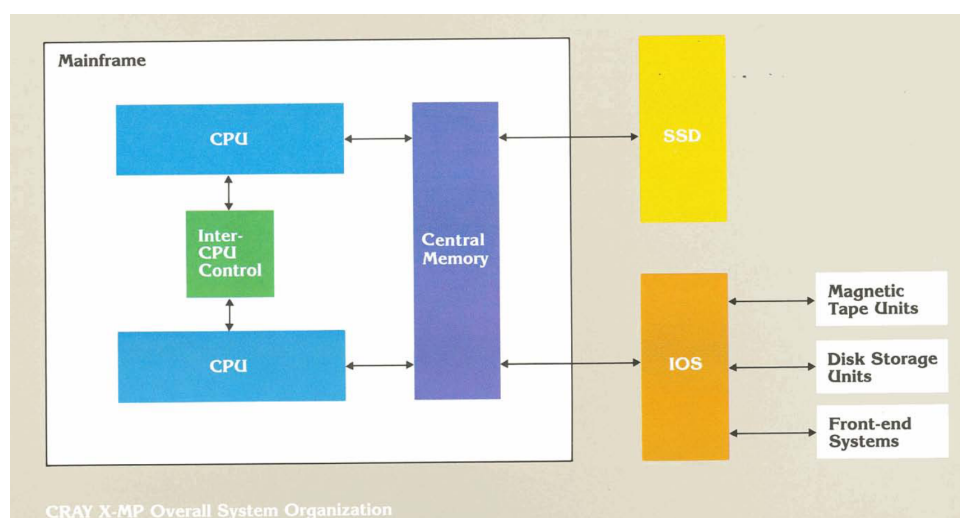


图 10: Cray X-MP

在软件层面，Cray X-MP 的创新硬件特性得到了 Cray Research 标准软件的支持。Cray 操作系统（COS）支持并发的独立单处理器任务以及单个任务的多处理。多处理可以通过 FORTRAN 程序启动和控制，同时也正在探索扩展 Cray FORTRAN 编译器（CFT）多处理能力的新技术 [10]。

在 X-MP 后，Cray Research 相继发布几款 PVP 结构的高性能处理器，其具体信息如表 3 所示 [5]。

发布时间	型号	最大 CPU 数	每个 CPU 的 CPU 速度大约峰值
1976	C1	1	0.160 GFLOPS
1982	XMP	4	0.235 GFLOPS
1988	YMP	8	0.333 GFLOPS
1991	C90	16	1 GFLOPS
1995	T90	32	2 GFLOPS

表 3: Cray 系列并行处理器性能表

银行 1 号

银河 1 号（银河系列）如图 11 所示高性能计算机是 1983 年中国自主研发的一款超级计算机，属于中国国家超级计算中心的高性能计算平台之一。银河 1 号在设计和性能上具有一定的创新性和领先性，主要用于科学计算、气象预测、工程模拟、数据处理等领域。

NEC SX-4

1994 年 11 月，NEC 在全球推出 SX-4 系列超级计算机如图 12 所示，其最大向量性能可达 1 teraFLOPS（每秒进行一万亿次浮点运算），采用 CMOS 和并行处理技术，性能卓越。在最大配置



图 11: 银河一号

下，SX-4 配备了 512 个并行 CPU，其向量性能可达到惊人的 1 teraFLOPS。这一成果得益于高密度超快 CMOS LSI（几何尺寸为 0.35 微米，集成约 400 万个晶体管）和高速 4 M bit 同步 SRAM 的应用。

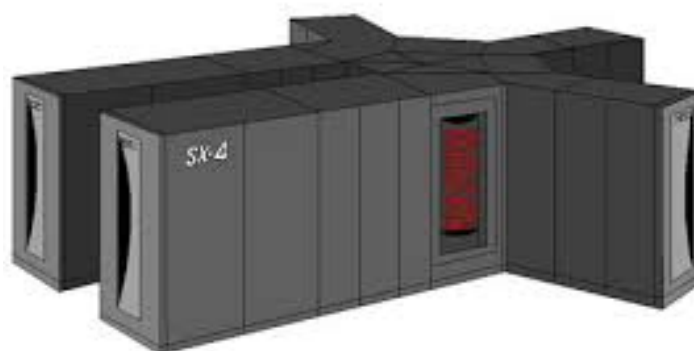


图 12: NEC SX-4

并行向量处理机未来发展趋势

并行向量处理机在高性能计算（HPC）领域具有显著优势。其强大的向量处理能力使得它能够高效处理大规模数据集，特别适用于科学计算、气候模拟、物理仿真和基因组学等计算密集型任务。通过对向量化操作的优化，PVP 能够实现极高的计算吞吐量，极大缩短计算时间。此外，PVP 还具备优异的并行性，能够通过多个处理单元同时执行复杂计算，充分发挥现代多核处理器的优势。

尽管并行向量处理机在许多领域表现出色，但其发展也面临一些不足和挑战。首先，PVP 系统通常对程序的向量化优化要求较高，需要专门的编程技术和工具支持，这可能增加了开发和维护的复杂性。此外，PVP 的硬件成本较高，尤其是在大规模部署时，能耗和散热问题也成为制约其发展的瓶颈。

从应用的角度看，PVP 通常需要与其他计算平台（如 GPU、FPGA 等）协同工作，构建异构计算系统，这要求开发人员具备更多的硬件与软件整合能力。

3.3 MPP（大规模并行处理器）

MPP（Massively Parallel Processing）是一种基于分布式计算的体系结构，旨在通过大量独立的计算节点并行执行计算任务。其核心思想是将一个大的数据处理任务分解为多个小的子任务，并在多个处理器上同时执行这些子任务，以实现高效的并行处理。每个节点都有独立的磁盘存储系统和内存系统，业务数据根据数据库模型和应用特点划分到各个节点上，每台数据节点通过专用网络或者商业通用网络互相连接，彼此协同计算。MPP 具有完全的可伸缩性、高可用、高性能、优秀的性价比、资源共享等优势。

MPP 的节点互联机制与 NUMA 不同。在单台物理服务器内部，MPP 通过高速互联实现处理器与本地内存的非一致性访问，并通过网络互联实现多个节点的分布式计算。其中，每个节点通常是一个独立的 SMP 系统。节点互联通常由专用硬和高速通信网络组成。这些计算节点通常拥有独立的内存，但节点间的通信延迟较低，通常采用专用的高速网络（如 InfiniBand）来减少通信开销。

MPP 将大型数据处理任务分解为多个小的子任务，并在多个节点上并行执行，从而显著提升数据处理速度。同时，数据在各个节点上分布式存储，使得每个节点能够独立处理其数据，并与其他节点协同工作。此外，MPP 架构采用分布式计算方式，每个节点独立计算一部分数据，最终汇总结果，这不仅提高了计算效率，还减少了单点故障的风险。其高并发性使得单个节点可以支持超过 300 个用户的并发访问，能够高效处理大规模数据集。随着数据规模的增长，MPP 架构支持通过增加节点数量进行横向扩展，展现出良好的可扩展性。最后，MPP 采用 Shared Nothing（完全无共享）架构，每个节点拥有独立的 CPU、内存和其他硬件资源，确保了系统的稳定性和可靠性。

MPP 的效率和性能取决于系统架构、处理器之间的通信以及软件工具的支持。MPP 的性能优化通常专注于提高通用计算能力，以处理多种计算密集型任务。它们需要在极高的浮点运算能力和大量内存带宽间取得平衡，关注的是如何快速并行处理大量数据。

MPP 系统通常使用高性能的 CPU、加速器（如 GPU）、大容量内存和快速存储系统，以便高效地处理数据密集型任务。文章中提到，这些硬件组件的选择和集成对于实现 MPPs 的高性能计算能力至关重要。

尽管 MPP 在当前的 HPC 领域中占有一席之地，但未来的系统架构可能会发展出新的并行处理模式，以应对技术趋势所带来的机遇和挑战。未来的 MPP 系统将需要在 bandwidth、overhead time 和 latency tolerance 方面进行权衡。

3.4 Cluster（集群）

计算集群通常指的是由多台独立的计算机（节点）组成的一个集合。这些计算机通过网络互联，共享任务来完成并行计算。计算集群的计算资源可以是同质的（所有节点具有相同硬件）或异质的（不同节点可能具有不同硬件配置）。集群系统通常使用分布式内存或共享内存（取决于集群的实现）。集群中的计算节点可以是低成本的标准服务器，通常配置为运行 Linux、Windows 或其他操

作系统。

节点互联集群系统通常依赖于标准硬件，而不是专用硬件。资源可能分散在多个物理机上，这些物理机通过标准网络连接（如千兆以太网、万兆以太网或 InfiniBand）进行通信。集群的资源管理通常依赖于操作系统和调度软件（如 Slurm、PBS、Torque、Hadoop 等）。计算集群的通信可能不像 MPP 那样具备专用的硬件支持，从而导致通信延迟相对较高，特别是在大规模集群中。

Cluster 的主要优点在于成本低，它基于普通的商用硬件，性价比高。同时，集群具有较高的灵活性，能够轻松扩展或替换节点，适应不断变化的需求。此外，集群的容错性也很强，单个节点的失效不会导致整个系统的瘫痪。然而，集群的管理复杂性较高，尤其是在大规模集群中，管理和维护任务包括处理节点故障、负载均衡和资源调度等，增加了运维难度。此外，集群中节点之间的通信依赖于网络带宽和延迟，在大规模集群中，通信开销可能成为瓶颈，影响整体性能。同时，在分布式存储系统中，如何确保数据的一致性和完整性也是一个重要问题，特别是在节点之间频繁进行数据交换的情况下，这为系统的稳定性带来了挑战。

3.5 Compute Express Link (CXL)

3.5.1 CXL 提出的背景

随着 NUMA 以及异构计算的普及，越来越多具有独立存储单元的计算设备被连接到计算机总线系统当中。而传统的 PCIe/DDR 对立的模式逐渐成为了系统的束缚。PCIe 和 DDR 在设计之初核心针对了两种不同意义上的扩展：设备的扩展以及存储的扩展。但随着 GPGPU、FPGA 等异构计算技术的发展，越来越多的 PCIe 设备开始具有自己独立的大容量存储器，但 PCIe 与 DDR 的对立使得这些局部于设备的大容量存储器难以被纳入到系统的缓存体系中。

而 CXL 协议的提出主要针对于为所有 PCIe 设备以及 DDR 设备提供进一步的系统层次抽象。CXL DMA 的引入可以解决传统 PCIe DMA 下无法被彻底解决而只能由软件层协调的数据不一致问题。由于这些不一致问题，软件设计工作者必须非常谨慎处理总线竞争信息，以保证系统可用，这种阻力限制了 PCIe 外设的进一步发展。CXL 的引入使得计算机系统可以更好地进行异构拓展，在 CXL 的抽象层之上，软件设计者可以几乎不考虑数据竞争地构建更加可靠的异构计算系统软件栈。

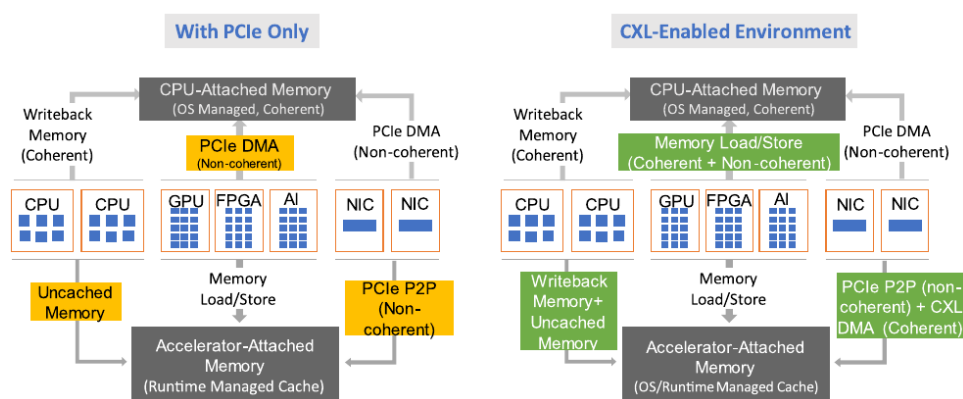


图 13: PCIe DMA 与 CXL DMA 的一致性对比

由于设备自带存储空间无法高效被整合到系统的可缓存地址空间中，当相应设备闲置时，整个系统常常无法调度该设备的额外存储空间，从而导致资源的闲置。因此，如何设计一种协议使得这些 PCIe 可以在闲置时让渡一部分存储器用于系统其他用途也是 CXL 设计的一个初衷。同时 CXL 也致力于实现更细粒度的 PIM 存储器共享。

3.5.2 CXL 的版本与历史沿革

Generation	Year	Scope	Speed	Use case
CXL 1.0 & CXL 1.1	2019	Single machine	32 GT/s	Accelerators (Challenge 1) Bandwidth and capacity expansion (Challenge 2)
CXL 2.0	2020	2–16 machines (single switch)	32 GT/s	Small-scale resource pooling (Challenge 3)
CXL 3.0	2022	100s machines (multiple switches)	64 GT/s	Large-scale resource pooling and sharing (Challenges 3 and 4)

图 14: CXL1.0 到 CXL3.0 的主要用途以及数据传输效率

随着 CXL 协议的提出，至今已经发展出了三代 CXL 版本。CXL 协议在代际保持向后兼容。在 CXL2.0 以及 CXL3.0 版本中，协议提出了带有多级交换技术的 CXL 以实现资源的跨主机共享。在 PCIe 的基础上，CXL 能够将 PIM 纳入到可缓存地址空间中。但 CXL 并不强制设备必须遵循 CXL DMA 的一致性数据访问，PCIe 设备仍然可以以非一致的方式 P2P 地访问其他设备的物理存储。

CXL1.01.1

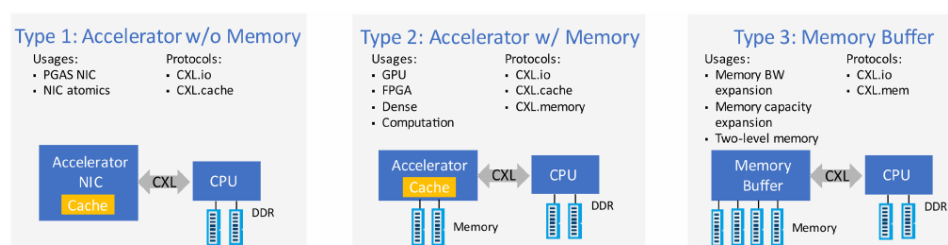


图 15: CXL1.0 引入的三种设备类别及其相关协议

针对 NIC、硬件加速设备、PCIe 拓展存储器，CXL 设计了 CXL.io、CXL.cache、CXL.memory 三个子协议。其中，CXL.io 为输入输出协议，CXL.io 基于 PCIe 协议实现了一套通用的设备发现以及初始化约定，并提供了对 DMA 的支持以及对直接 PCIe 原生接口的协议化以便于旧的软件栈能够在 CXL 上兼容。

CXL.cache 的提出主要为了提供一套简洁的可拓展缓存一致性协议。基于 MESI 协议、CXL.cache 可以将外部设备的存储器纳入到系统可缓存地址空间中。同时，经由 CXL.cache，PCIe 设备之间不再进行直接的对等缓存交互，以便于系统跟踪其一致性。

CXL.memory 协议用于要求设备让渡自身存储器的管理权，从而将其转让给主机管理，即 HDM。当设备存储器以 HDM 的形式暴露给系统时，系统将可以使用类似 DDR 内存的管理方

式管理全部设备空间内存。CXL.memory 支持了 HDM-H 和 HDM-D 两种管理方式。其中 HDM-H 模式下，缓存失效必须由主机侧发起，而 HDM-D 模式下缓存失效可以由设备主动发起。由于 HDM-H 模式下设备不需要维护缓存状态信息，因而具有更小的设备间通信开销，而 HDM-D 则允许设备介入系统的一致性，从而有利于提供更细粒度的缓存一致性。

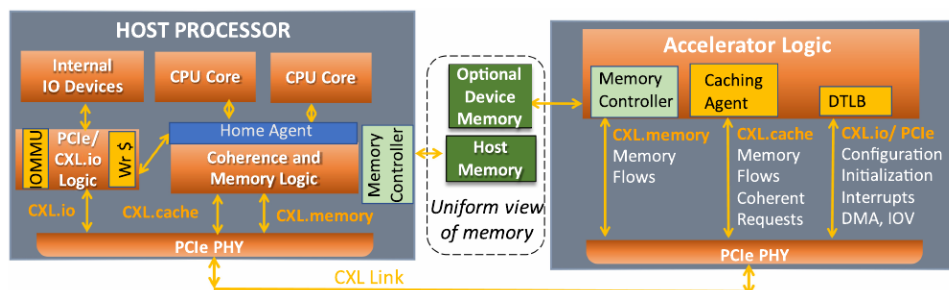


图 16: 子协议的工作位置

CXL2.0 以及 CXL3.0

在 CXL2.0 中引入了资源池、以及 QoS 的概念，其中资源池可以解决前文中提到的闲置资源利用率地下的问题，QoS 通过源节流控制技术可以根据不同服务的需求以及重要程度分配不同的 PCIe 带宽。除此之外 CXL2.0 中引入了 CXL 交换机技术以及热插拔技术使得系统的扩展性与动态性得到了进一步改善。相较于 CXL1.1 只适用于单机系统，CXL2.0 及之后版本均可以借助 CXL 交换机实现 2~16 个主机间互联的片上系统。此外，设备也被进一步秋分为 SLD 和 MLD 以使得不同主机可以分别共享一个设备的一部分。

CXL3.0 通过多机 CXL 交换实现了至多可达 4096 台主机的互联，使得该系统可以被用于大规模分布式系统，并在这样的系统中维持数据的一致性。在大规模分布式系统中，由于中央控制一旦存在就很容易成为瓶颈，因此点对点访问的协议支持变得尤为重要。CXL3.0 的点对点访问机制主要可以体现在 PBR 机制以及 CXL 交换的 Fabric 拓扑支持。

3.6 集群虚拟化技术

在高性能计算（HPC, High Performance Computing）领域，集群（Cluster）虚拟化技术的使用与传统的数据中心或企业虚拟化有所不同。HPC 集群侧重于通过多个计算节点协同工作来处理需要大量计算资源的任务，因此虚拟化在这种环境中的应用要特别考虑性能、资源利用率和低延迟等因素。以下是一些在 HPC 集群中使用的虚拟化技术及其特点。

3.6.1 虚拟化对 HPC 的影响

在传统的 HPC 集群中，虚拟化技术常常被认为是一个负担，因为虚拟化层会引入额外的开销，影响计算密集型应用的性能。然而，随着技术的发展，虚拟化在 HPC 中的使用场景逐渐增多，尤其是在以下几个方面：

- 资源隔离与管理：虚拟化能够有效地隔离不同用户或任务的资源，尤其是在共享集群的多租户环境中。
- 弹性扩展与动态资源调度：虚拟化技术可以通过动态分配计算资源来应对工作负载波动，从而提高集群的灵活性和资源利用率。

- 集群管理与维护：虚拟化技术可以简化集群的管理和维护，比如简化节点的故障恢复、系统升级等。

3.6.2 常见的虚拟化技术

以下是一些高性能计算集群中常见的虚拟化技术：

全虚拟化与半虚拟化

- 全虚拟化 (Full Virtualization) 技术在虚拟机中运行完整的操作系统，并完全模拟硬件。这种方式的性能开销较大，通常不适合对性能要求极高的 HPC 场景。
- 半虚拟化 (Para-Virtualization) 则通过与操作系统内核的合作，减少虚拟化的开销。虽然它对性能的影响较小，但需要对操作系统进行修改。

容器虚拟化 (Containerization)

容器虚拟化是近年来在 HPC 领域逐渐流行的技术。与传统虚拟机不同，容器不需要模拟硬件，且共享宿主操作系统的内核，因而更加轻量。容器技术在 HPC 中的应用可以提供更低的开销、更好的资源利用率和更高的灵活性。常见的容器虚拟化技术有：

- Docker：通过封装应用及其依赖，提供轻量的隔离环境，适用于部署高效、可移植的 HPC 应用。
- Singularity：专为 HPC 和科学计算设计的容器化工具，它支持在 HPC 集群中运行容器，避免了对虚拟机带来的性能损失，同时提供与 Docker 类似的容器化功能。Singularity 能够与现有的 HPC 调度系统（如 SLURM）无缝集成，是 HPC 环境中的热门选择。
- Podman：与 Docker 类似，但它不依赖于守护进程，因此在某些集群环境中具有更高的安全性和效率。

虚拟机 (VM) 虚拟化

- KVM (Kernel-based Virtual Machine)：KVM 是 Linux 内核的一个虚拟化模块，允许将物理服务器虚拟化为多个虚拟机。KVM 在 HPC 环境中的使用通常比较有限，因为它的性能开销较大，但它仍然可以用于需要虚拟化隔离的应用。
- Xen：Xen 是一个高性能的虚拟化解决方案，提供全虚拟化和半虚拟化支持。它在需要高性能和可扩展性的 HPC 应用中得到了较为广泛的使用。
- VMware ESXi：虽然 VMware 在传统数据中心中被广泛使用，但它在 HPC 集群中的应用较为有限，因为它也会引入虚拟化开销，通常在需要较强资源隔离或多租户支持时使用。

高效的虚拟化技术：虚拟化网络和存储

在 HPC 集群中，存储和网络的虚拟化至关重要，因为高效的数据传输和存储管理是提高性能的关键。以下是常见的虚拟化技术：

- 软件定义存储 (SDS)：通过虚拟化存储资源，动态分配和管理存储空间。常见的 SDS 技术有 Ceph、GlusterFS 等。
- 虚拟化网络 (SDN)：软件定义网络可以帮助更灵活地管理和调度 HPC 集群中的网络资源，特别是在集群规模较大时，SDN 能够实现高效的网络流量管理和负载均衡。

虚拟化在 HPC 中的挑战

尽管虚拟化技术在 HPC 集群中具有一些优势，但也面临一些挑战，主要包括：

- 性能开销：虚拟化技术引入的额外层次和资源分配可能导致性能下降。对于计算密集型任务，虚拟化可能不适用，尤其是在需要充分利用 CPU 和内存资源时。
- I/O 瓶颈：虚拟化对存储和网络 I/O 性能的影响尤其显著。HPC 应用往往依赖高速数据传输，虚拟化可能带来网络和存储的延迟。
- 兼容性问题：某些 HPC 应用可能不支持虚拟化环境，或者需要进行特殊的配置和优化才能正常运行。

虚拟化技术的优化与增强

为了减少虚拟化对 HPC 性能的影响，业界已经进行了一些优化和增强：

- 硬件虚拟化支持：现代的处理器的（如 Intel 的 VT-x、AMD 的 AMD-V）提供硬件级的虚拟化支持，可以显著减少虚拟化开销。
- NUMA 优化：在大规模并行计算中，NUMA（非统一内存访问）架构至关重要。虚拟化技术需要优化内存访问路径，以避免性能瓶颈。
- GPU 虚拟化：随着 GPU 计算在 HPC 中的普及，GPU 虚拟化技术也在发展。NVIDIA 的 **nvGPU**（虚拟 GPU）允许多个虚拟机共享 GPU 资源，有助于提高图形计算的并行性。

HPC 虚拟化管理平台

一些虚拟化平台专门为 HPC 设计，提供集群资源的管理和调度功能。常见的 HPC 虚拟化管理平台有：

- OpenStack：作为一个开源的云计算平台，OpenStack 能够管理计算、存储和网络资源。通过与 HPC 调度系统（如 Slurm）的集成，OpenStack 可以实现对 HPC 集群的虚拟化管理。
- Slurm：Slurm 是广泛使用的 HPC 集群调度系统，它可以与虚拟化技术（如 KVM、Docker）集成，支持虚拟化资源的调度和管理。
- PBS (Portable Batch System)：另一个广泛应用于 HPC 集群管理的调度系统，支持虚拟化技术的集成和资源管理。

总结

在 HPC 领域，虚拟化技术正在逐渐被接受并应用于资源隔离、弹性扩展和集群管理等方面。尽管虚拟化在性能上可能带来一定的开销，但通过容器化技术（如 Singularity、Docker）、硬件虚拟化支持、GPU 虚拟化等方式，HPC 集群中的虚拟化性能得到了显著提高。随着技术的进步，虚拟化技术在 HPC 集群中的应用前景将更加广阔，尤其是在多租户环境、弹性计算、云计算和大数据分析等领域。

3.7 Cluster 与 MPP 硬件架构比较

3.7.1 Cluster 架构硬件架构

Cluster 架构是一种分布式计算环境，通过多个节点共同完成计算任务。这些节点通过高速网络互联，共享数据和任务。

核心特点

- 共享内存与共享磁盘架构：Cluster 通常不共享内存，每个节点都有自己的内存。共享磁盘架构（如网络文件系统 NFS）可以通过集中化的存储设备提升数据一致性，但可能存在 I/O 瓶颈。
- 负载均衡：Cluster 系统通过调度算法实现动态负载均衡，提高资源利用率。
- 扩展性：Cluster 可以通过增加节点数量实现横向扩展，但网络带宽和节点间通信复杂度可能成为瓶颈。
- 连接网络：以太网和 InfiniBand 是 Cluster 系统的常见选择，而 NVLink 和 PCIe 更多用于节点内部的高速连接，尤其是在 GPU 加速的环境中。

数据迁移与键的分布

- Cluster 支持在线数据迁移，即在不停止服务的情况下，可以将数据从一个节点迁移到另一个节点。
- 使用哈希标签确保相关的键被映射到同一个哈希槽，适合使用扁平化的数据模型。

持久化与集群管理

- Cluster 支持 RDB 和 AOF 持久化方式，每个节点独立写入自己的 AOF 文件。
- 集群创建、节点管理、故障诊断都可以通过特定的命令和工具进行，如使用 redis-cli 工具的 cluster 命令创建集群。

客户端支持与限制

- 大多数 Redis 客户端都支持 Cluster 模式，客户端库会处理请求的路由和重试逻辑。
- 在 Cluster 模式下，事务和 Lua 脚本只能在一个哈希槽内操作，键空间通知在 Cluster 模式下不可用。

故障切换与高可用性

- Cluster 架构中的 Sentinel 系统可以提供高可用性解决方案，通过监控主节点并在主节点故障时自动进行故障切换。

学术研究与应用

- 2022 年 IEEE Cluster 会议强调了集群系统设计、软件堆栈能力提升、系统管理和监控以及算法、方法和应用的设计等多个方面的研究。
- 会议涵盖了 HPC 和大数据应用在大规模集群上的研究、HPC 和大数据集群的架构、能源效率集群架构等议题。

3.7.2 MPP 架构硬件架构

MPP（大规模并行处理）架构通过将计算任务分解成多个子任务并行执行，以实现高效的数据处理能力。每个处理节点独立完成其任务，并通过高速网络协调。

核心特点

- 自动数据分片：MPP 系统会将大规模数据集切分成多个小块，并分配到不同节点进行处理。
- 节点独立性：每个节点都有自己的处理器、内存和存储，避免了共享存储带来的瓶颈。
- 高并行度：通过同时运行多个任务，大幅度提高了计算效率。
- 互联网络体系：MPP 系统依赖于低延迟、高带宽的网络连接，例如光纤以太网和 InfiniBand，以支持节点间的高效通信。

共享无关架构 (Shared-nothing Architecture)

- MPP 系统中的每个节点通常有独立的 CPU、内存、存储和网络资源，这种架构被称为“共享无关架构”。

线性可扩展性

- MPP 系统可以通过增加节点数量来线性提升系统的处理能力。

与传统架构的对比

- 与 SMP 相比，MPP 每个节点独立处理任务，拥有自己的存储和内存资源，不存在资源争夺问题，因而可扩展性强。
- 与 Hadoop/Spark 相比，MPP 擅长处理结构化数据，特别是 SQL 数据查询和大规模数据仓库场景中，性能往往优于 Hadoop 和 Spark。

技术特点

- MPP 架构采用列存储、透明压缩和智能索引技术，为海量数据分析型应用领域提供极高性能。
- MPP 架构通过并行加载机制和先进灵活的数据分布策略实现数据向集群的均匀分布加载，避免热节点或数据倾斜。

典型应用场景

- 数据仓库：MPP 系统是现代数据仓库的基础架构，支持复杂的 SQL 查询优化。
- 商业智能：如 AWS Redshift、Google BigQuery 提供基于 MPP 的快速数据分析能力。

一篇论文提出了一种新的处理解决方案，描述了一个名为“Whale”的系统，它使用 MPP 无共享架构通过分布式 SQL 执行实现了海量数据的有效信息提取。该系统后端使用 MySQL 作为分析引擎，通过上层节点并行控制引擎的执行，并应用分布式文件结构使系统存储具有可靠性。这种设计合理且轻量级的架构，使用 MySQL 的不同存储引擎，使得在不同场景下对海量数据的处理都能得到满足。

MPP 架构在轨道交通大数据中心的应用

一项研究分析了城市轨道交通大数据中心建设中面临的挑战，并介绍了基于 Hadoop+MPP 技术架构的大数据中心建设方案。该方案利用 MPP 架构的高性能和可扩展性，实现了对海量数据的有效处理和分析。例如，苏州轨道交通项目实践表明，基于 Hadoop+MPP 架构的大数据中心建设方案能够达到预期效果。

未来发展方向

未来 MPP 数据库的发展方向包括智能化、混合存储、云原生、实时分析和多模数据处理。例如，随着人工智能技术的发展，越来越多的 MPP 数据库开始集成机器学习算法，支持自动优化查询性能和辅助数据分析。

3.7.3 Cluster 与 MPP 对比

扩展性：

- Cluster 系统通过增加节点数量实现横向扩展，但网络延迟和通信复杂度限制了性能提升。
- MPP 系统通过任务分片实现了高度并行化，对硬件资源的依赖更高。

性能：

- Cluster 更适合小规模、高通信需求的任务，例如分布式文件存储。
- MPP 更适合复杂查询和大规模数据分析，例如实时数据仓库查询。

成本：

- Cluster 的构建成本相对较低，适合通用计算任务。
- MPP 架构由于需要高性能硬件和优化的软件支持，成本更高。

两者的区别在于，MPP 系统往往设计上更加集成、专用，追求极致的并行计算性能，而集群系统则更多依赖通用计算节点，通过灵活的资源管理实现分布式计算，适用范围更广。表 4 给出了一个更具体的 MPP 与 Cluster 的特性对比。

特性	MPP	Cluster
架构	大规模并行计算专用硬件，紧密的集成结点，优化通信	由多台独立的计算机组成，资源可以共享或者独立
资源管理	专用硬件，优化的资源调度和任务调度	标准硬件，有很多灵活的资源调度工具（Slurm、PBS 等）
通信方式	高效的告诉网络（如 Infini-Band）连接，低延迟通信	通过标准网络连接，通信开销更大，依赖于调度软件
扩展性	有限，增加节点需要专门的硬件支持	良好，扩展非常灵活，增加节点较为容易
应用场景	高性能计算、科学计算、工程计算等	大数据处理、Web 服务、虚拟化、大规模并行任务等

表 4: MPP 与 Cluster 的对比

3.7.4 总结

Cluster 和 MPP 架构在现代计算系统中各有优势。Cluster 更注重灵活性和节点间的协作，适合多种通用计算任务；而 MPP 则以高度并行化的数据处理能力，成为数据仓库和大数据分析的主力架构。未来，这两种架构可能在异构计算、智能调度和网络优化方面进一步融合与发展。

4 总结

表 5 汇总了本文调研各类体系结构时所收集到的具体的超算实例信息以及相关参考文献，图 17 给出了这些信息的一个相对直观的展示。分析超级计算机的历史发展脉络，我们不难发现，在超级计算机出现早期，研究工作核心关注于 PVP 系统的设计。PVP 系统致力于构建高性能的单机向量处理系统。而随着单芯片集成度逐渐达到量子极限，研究工作的重心逐渐从高效的单机系统向多机协同系统转移。

从客观上来讲，多种力量正促进着这种转移的出现。一方面，商业化芯片的日臻成熟促成了一大批 Cluster 集群步入人们的视野，构建多机系统的成本逐渐降低，这使得设计专用的硬件产品显得过于费时费力。另一方面，计算机网络的发展也进一步促进了分布式系统的发展与完善，使得原先难以被分布式完成的任务，可以在相对高效的光纤以太网中合理且高效地完成数据分发工作。

但调度、通信以及多机管理对于 Cluster 在性能领域的长足进步仍然颇有限制。出于这种限制，我们可以推测，MPP 与 Cluster 在超算领域“平分天下”的格局在短时间内很难有显著的变化。MPP 因其极高的性能在科研领域发挥重要作用，为理论与工程研究提供重要支撑，但由于其硬件的专用性以及资源的集中性，因而常常不利于市场化与商业化的普及。而 Cluster 系统依托各大软件厂商的云服务平台，为其他企业或个人提供易于灵活拓展的计算服务。

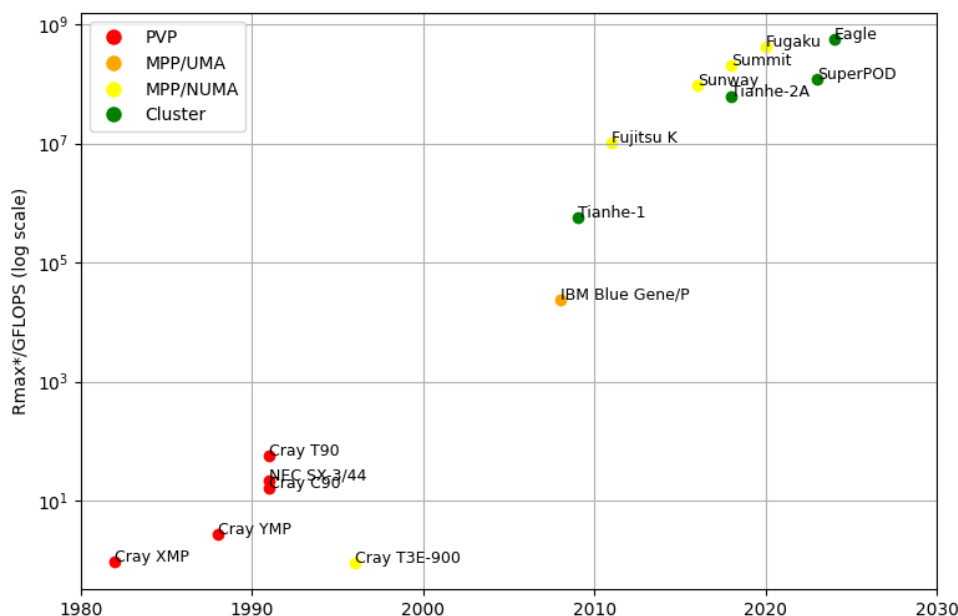


图 17: 文中提及的所有高性能计算机的性能图表

名称	架构分类	建成年份	等效 Rmax	数据来源
Cray XMP	PVP	1982	$0.235 \text{ GFLOP} \times 4^-$	[5]
Cray YMP	PVP	1988	$0.333 \text{ GFLOP} \times 8^-$	[5]
Cray C90	PVP	1991	$1 \text{ GFLOP} \times 16^-$	[5]
Cray T90	PVP	1991	$1.8 \text{ GFLOP} \times 32^-$	[5]
NEC SX-3/44	PVP	1991	22 GFLOPS	[23][13]
Tianhe-1	Cluster	2009	563.10 TFLOPS/s	[24]
Tianhe-2A	Cluster	2018	61.44 PFLOPS	[25]
Eos NVIDIA DGX SuperPOD	Cluster	2023	121.40 PFLOPS	[18]
Eagle	Cluster	2023	561.2 PFLOPS	[17]
Cray T3E-900	MPP/NCC-NUMA	1996	0.9 GFLOPS	[3]
IBM Blue Gene/P	MPP/UMA	2008	23.86 TFLOPS	[16]
Fujitsu K	MPP/Tofu(NUMA)	2011	10.5 PFLOPS	[19]
Fugaku	MPP/CC-NUMA	2020	442.01 PFLOPS	[22]
Summit	MPP/Hybrid(NUMA)	2018	200 PFLOPS	[20]
Sunway TaihuLight	MPP/CC-NUMA	2016	93.01 PFLOPS	[21]

表 5: 超算信息及其参考资料

5 分工情况以及参考文献

5.1 小组分工情况

小组成员	学号	负责内容
宋震	ZB2406316	SIMD: 向量流水机、阵列处理器, GPU 以及 Tensor Core
苏晓	BY2406130	SMP、CC-NUMA, 虚拟化技术
张泰然	ZY2406337	MIMD: PVP, NPU-IANUS
刘旭昭	SY2406327	MIMD: MPP、Cluster 以及其硬件架构对比
郭冠男	SY2406410	CXL 相关内容, 编写概述, 汇总, 制作 PPT 及文档

表 6: 分工表

参考文献

- [1] Wikipedia contributors. *Advanced Vector Extensions*. Accessed: 2024-11-30. 2024. URL: https://en.wikipedia.org/wiki/Advanced_Vector_Extensions.
- [2] Wikipedia contributors. *ARM architecture family*. Accessed: 2024-11-30. 2024. URL: https://en.wikipedia.org/wiki/ARM_architecture_family.
- [3] Wikipedia contributors. *Cray T3E*. Accessed: 2024-11-30. 2024. URL: https://en.wikipedia.org/wiki/Cray_T3E.

- [4] Wikipedia contributors. *Seymour Cray*. Accessed: 2024-11-30. 2024. URL: https://en.wikipedia.org/wiki/Seymour_Cray.
- [5] Cray-History.net. *PVP Generations, XMP, YMP, C90, T90*. Accessed: 2024-11-30. 2024. URL: <https://cray-history.net/faq-1-cray-supercomputer%20families/#T0C3>.
- [6] David Culler, Jaswinder Pal Singh, and Anoop Gupta. *Parallel Computer Architecture: A Hardware/Software Approach*. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1998. ISBN: 9780080573076.
- [7] M.J. Flynn. “Very high-speed computing systems”. In: *Proceedings of the IEEE* 54.12 (1966), pp. 1901–1909. DOI: 10.1109/PROC.1966.5273.
- [8] Yong Fu. *Performance Optimization on Modern Processor Architecture through Vectorization*. Accessed: 2024-11-30. 2016. URL: <https://objectcomputing.com/resources/publications/sett/december-2016-performance-optimization-on-modern-processor-architecture-through-vectorization>.
- [9] Daniel Gerzhoy et al. “Nested MIMD-SIMD Parallelization for Heterogeneous Microprocessors”. In: 16.4 (Dec. 2019). ISSN: 1544-3566. DOI: 10.1145/3368304. URL: <https://doi.org/10.1145/3368304>.
- [10] cray-rearch inc. *The CRAY X-MP Series of Computers*. Accessed: 2024-11-30. 1983. URL: <http://s3data.computerhistory.org/brochures/cray.x-mp.1983.102646267.pdf>.
- [11] Norman P Jouppi et al. “In-datacenter performance analysis of a tensor processing unit”. In: *Proceedings of the 44th annual international symposium on computer architecture*. 2017, pp. 1–12.
- [12] Norman P Jouppi et al. “Ten lessons from three generations shaped google’ s tpuv4i: Industrial product”. In: *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE. 2021, pp. 1–14.
- [13] museum.ipsj.or.jp. **【NEC】 SX-4**. Accessed: 2024-11-30. 2024. URL: <https://museum.ipsj.or.jp/en/computer/super/0018.html>.
- [14] Eriko Nurvitadhi. *Real Performance of FPGAs Tops GPUs in the Race to Accelerate AI*. Accessed: 2024-11-30. 2020. URL: <https://www.intel.com/content/dam/www/central-libraries/us/en/documents/real-performance-of-fpgas-tops-gpus-in-the-race-to-accelerate-ai-whitepaper.pdf>.
- [15] John Shen and Mikko Lipasti. *Modern Processor Design: Fundamentals of Superscalar Processors*. Morgan Kaufmann, 2005.
- [16] top500.org. *Blue Gene/P Solution*. Accessed: 2024-11-30. 2008. URL: <https://www.top500.org/system/176518/>.
- [17] top500.org. *Eagle - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR*. Accessed: 2024-11-30. 2023. URL: <https://top500.org/system/180236/>.
- [18] top500.org. *Eos NVIDIA DGX SuperPOD - NVIDIA DGX H100, Xeon Platinum 8480C 56C 3.8GHz, NVIDIA H100, Infiniband NDR400*. Accessed: 2024-11-30. 2023. URL: <https://www.top500.org/system/180239/>.

- [19] top500.org. *K computer, SPARC64 VIIIfx 2.0GHz, Tofu interconnect*. Accessed: 2024-11-30. 2011. URL: <https://top500.org/system/177232/>.
- [20] top500.org. *Summit - IBM Power System AC922, IBM POWER9 22C 3.07GHz, NVIDIA Volta GV100, Dual-rail Mellanox EDR Infiniband*. Accessed: 2024-11-30. 2018. URL: <https://top500.org/system/179397/>.
- [21] top500.org. *Sunway TaihuLight - Sunway MPP, Sunway SW26010 260C 1.45GHz, Sunway*. Accessed: 2024-11-30. 2016. URL: <https://www.top500.org/system/178764/>.
- [22] top500.org. *Supercomputer Fugaku - Supercomputer Fugaku, A64FX 48C 2.2GHz, Tofu interconnect D*. Accessed: 2024-11-30. 2020. URL: <https://www.top500.org/system/179807/>.
- [23] top500.org. *SX-3/44*. Accessed: 2024-11-30. 1997. URL: <https://www.top500.org/system/170812/>.
- [24] top500.org. *Tianhe-1 - NUDT TH-1 Cluster, Xeon E5540/E5450, ATI Radeon HD 4870 2, Infiniband*. Accessed: 2024-11-30. 2008. URL: <https://top500.org/system/176546/>.
- [25] top500.org. *Tianhe-2A - TH-IVB-FEP Cluster, Intel Xeon E5-2692v2 12C 2.2GHz, TH Express-2, Matrix-2000*. Accessed: 2024-11-30. 2018. URL: <https://top500.org/system/177999/>.
- [26] 中存储. 从超算榜 TOP500 看 Cluster 和 MPP 架构在超算系统中的实际应用. Accessed: 2024-11-30. 2018. URL: <https://wap.chinastor.com/hpc-top500/06303SD2018.html>.